

SIVF: Streaming Inverted File Indexing on GPUs

Dongfang Zhao

dzhao@uw.edu

HPDIC Lab: <https://hpdic.github.io/>

University of Washington, USA

Abstract

1 Introduction

1.1 Observation: Deletion is Expensive

Real-world vector search applications, such as real-time recommendation and fraud detection, increasingly operate on streaming data where timeliness is critical. These systems require a sliding window model where expired vectors must be evicted as new vectors arrive to maintain bounded memory usage. However, existing GPU-accelerated approximate nearest neighbors (ANN) indices are predominantly optimized for write-once-read-many workloads.

To quantify the gap between insertion and eviction performance, we conducted benchmarks using both the Python (precompiled package through PyPI) and C++ (local compilation from source) interfaces of Faiss [2] on an NVIDIA RTX 6000 GPU with the SIFT1M dataset [3] on the Chameleon testbed [4]. As illustrated in Figure 1, we observe a severe performance asymmetry across both implementations. In the native C++ benchmark, while inserting a batch of 10,000 vectors takes only 28.6 ms due to efficient GPU parallelism, evicting the same number of vectors incurs a latency of 202.2 ms. This constitutes a ~ 7.1 times slowdown, confirming that the bottleneck is fundamental to the index design rather than language binding overhead.

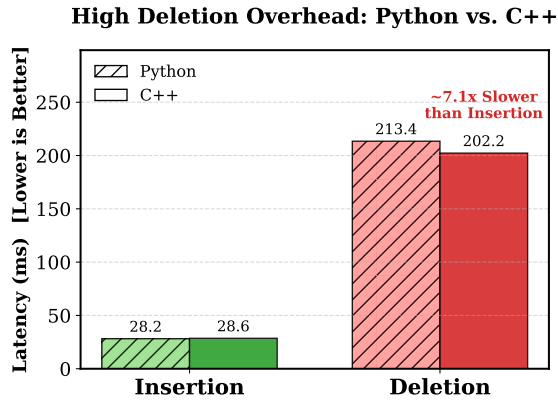


Figure 1: The High Cost of Deletion compared across Python and C++ interfaces. Even using the native C++ implementation, while GPU parallelism accelerates vector insertion (~ 28.6 ms), the lack of in-place GPU eviction support causes deletion latency to surge to ~ 202.2 ms. This represents a ~ 7.1 x slowdown, confirming this asymmetry as the primary bottleneck in streaming scenarios regardless of the language binding.

1.2 Root Cause: The Missing Operator

Our analysis of the Faiss source code [1] reveals that the performance asymmetry shown in Figure 1 is not merely an optimization issue, but a fundamental feature gap enforced by the library’s class hierarchy.

In the Faiss codebase, the data eviction interface is defined in the abstract base class `faiss::Index` via the `remove_ids` virtual method. By default, this base implementation throws an exception, indicating a lack of support. While CPU-based implementations such as `IndexIVF` and `IndexFlat` explicitly override this method to provide efficient, in-memory deletion logic (e.g., via `std::vector` manipulation or `memmove`), their GPU counterparts fail to do so. Specifically, the GPU base class `faiss::gpu::GpuIndex` and its derivatives (e.g., `GpuIndexIVF`) inherit the exception-throwing base implementation without providing a GPU-native override.

This implementation gap necessitates a fallback to a distinct *CPU-GPU Roundtrip* pattern for eviction, as observed in our benchmarks. Consequently, to delete even a single vector from a GPU-resident index, the system must transfer the entire index state to host memory, perform the deletion using CPU primitives, and re-upload the modified index to the device.

1.3 Technical Challenges

The absence of a GPU-native `remove_ids` stems from the complexity of managing dynamic data structures in VRAM. Unlike the CPU’s `IndexIVFFlat`, which utilizes dynamic arrays (STL vectors) to manage inverted lists, GPU indices rely on pre-allocated, contiguous memory blocks to maximize memory bandwidth and parallelism. Implementing in-place deletion on the GPU would require:

- **Dynamic Memory Management:** Frequent reallocation and compaction of inverted lists within VRAM to prevent fragmentation.
- **Complex Synchronization:** Fine-grained locking mechanisms to prevent race conditions when thousands of concurrent GPU threads attempt to modify shared list structures.
- **Reverse Mapping Overhead:** Standard IVF indices do not maintain an ID-to-Location mapping. Locating a specific ID for deletion requires scanning all inverted lists, an operation with $O(N)$ complexity that negates the benefits of GPU acceleration.

These architectural constraints render in-place GPU eviction prohibitively complex to implement efficiently, forcing the current IO-bound workaround.

1.4 Proposed Solution: The SIVF Architecture

To overcome the architectural hurdles identified in Section 1.3, we propose SIVF (Streaming Inverted File), a new GPU indexing

architecture designed specifically for high-throughput insertion and deletion, and eventually for efficient streaming data analytics on GPUs. SIVF introduces three key mechanisms to resolve the identified bottlenecks:

Slab-based Memory Management. To mitigate the overhead of **Dynamic Memory Management**, SIVF abandons the contiguous memory layout of standard IVF. Instead, we propose a *Slab Allocation System*. The GPU memory is pre-allocated as a pool of fixed-size blocks (e.g., 4KB pages). Inverted lists are implemented as chains of these blocks. This design obviates the need for costly memory reallocation and data compaction. When a list grows, a new block is simply linked from the free pool using atomic operations. This ensures that memory fragmentation is virtually eliminated and ingestion latency remains constant regardless of list size.

Bitmap-based Lazy Eviction. To address **Complex Synchronization**, we decouple logical deletion from physical data removal. We introduce a *Validity Bitmap* associated with each memory block. Eviction is implemented as a *Lazy Deletion* strategy: removing a vector involves only a single atomic bit-flip in the bitmap to mark the slot as invalid. This lock-free approach allows thousands of concurrent threads to perform deletions without contention. Physical reclamation of memory blocks is deferred and performed asynchronously by a background kernel only when the utilization of a block drops below a threshold, effectively hiding the cleanup cost.

GPU-Resident Address Table. To eliminate the **Reverse Mapping Overhead**, SIVF maintains a compact, GPU-resident *Address Translation Table* (ATT). This table maps each active Vector ID to its precise physical location (composed of BlockID and SlotOffset). By compressing this location metadata into a single 64-bit integer, we minimize the memory footprint. This structure allows the eviction kernel to locate any target vector in $O(1)$ time, avoiding the prohibitive $O(N)$ scan required by current libraries [1].

By synthesizing these three techniques, SIVF enables true in-place modification of the index directly on the GPU, breaking the dependency on the host CPU and eliminating the roundtrip bottleneck. As a result, this paves the road for high-throughput, low-latency streaming vector search applications on GPU platforms.

1.5 Contributions

In summary, this paper makes the following contributions:

- We identify a critical performance bottleneck in existing GPU-accelerated ANN indices stemming from the lack of native support for in-place data eviction. We analyze the root causes of this limitation and quantify its impact through empirical benchmarks.
- We propose SIVF, a novel indexing architecture that enables efficient streaming insertion and deletion directly on the GPU. SIVF introduces slab-based memory management, bitmap-based lazy eviction, and a GPU-resident address translation table to overcome the challenges of dynamic data structures in VRAM.
- We implement a prototype of SIVF and evaluate its performance against state-of-the-art GPU ANN libraries. Our

results demonstrate that SIVF significantly reduces eviction latency, paving the way for real-time streaming vector search applications on GPUs.

2 Related Work

3 Design and Implementation

3.1 Slab-based Memory Management

Standard GPU-based IVF indices rely on contiguous memory arrays to store inverted lists. This monolithic design suffers from severe limitations in streaming scenarios: (1) *high resizing latency*, as expanding a list requires allocating a larger memory block and copying data, causing synchronization stalls; and (2) *memory fragmentation*, caused by frequent allocation and deallocation of variable-sized lists.

To address these challenges, we propose a Slab-based Dynamic Memory Allocator (SDMA). Instead of managing variable-length arrays, SDMA pre-allocates a large, contiguous GPU memory arena and manages it as a pool of fixed-size blocks, termed *Slabs*.

3.1.1 Slab Layout and Metadata. We define a Slab as the fundamental unit of storage. Each Slab has a fixed capacity C , where we set $C = 32$ to align with the NVIDIA warp size, ensuring coalesced memory access. A Slab S_i consists of two distinct regions:

- **Data Payload:** Stores the quantized vectors. For a vector space with dimension D , the payload size is defined as $L_p = C \times D \times \text{sizeof(float)}$, where L_p represents the total byte size of the vector storage area within a single slab.
- **Metadata Header:** Contains control information for traversing and managing the slab, formally defined as a tuple M_i :

$$M_i = \langle n_{next}, b_{valid}, c_{valid} \rangle \quad (1)$$

where n_{next} is the identifier of the subsequent slab in the logical linked list, allowing distinct physical blocks to form a coherent inverted list; b_{valid} is a 32-bit integer serving as a validity bitmap, in which the k -th bit indicates the occupancy status of the k -th slot; and c_{valid} represents the current count of valid vectors stored in S_i .

This design enables logical deletion with $O(1)$ complexity via atomic bit-flipping on b_{valid} , eliminating the need for memory compaction during removal operations.

3.1.2 Kernel-level Lock-free Allocator. Traditional memory management requires host-device communication (e.g., `cudaMalloc`), which incurs microsecond-level latency that is unacceptable for high-throughput streaming. We implement a *device-side allocator* that enables GPU kernels to request and release memory directly.

The allocator manages a global *Free List*, implemented as a pre-allocated stack in global memory. We utilize a strictly atomic approach to ensure thread safety without mutex locks:

- **Allocation:** A thread (or warp) requests a new Slab by atomically decrementing the global stack pointer P_{top} . The operation is modeled as:

$$idx = \text{atomicSub}(P_{top}, 1) - 1 \quad (2)$$

where idx denotes the index in the free list array, and the allocator returns the Slab ID stored at `FreeList[idx]`.

- *Deallocation*: To release a Slab, the thread atomically increments P_{top} and pushes the Slab ID back onto the stack structure.

Both operations achieve $O(1)$ time complexity and bypass the OS-level memory manager entirely, resulting in near-zero allocation overhead.

3.1.3 Virtual Addressing via Address Table. To decouple the logical vector ID from its physical storage location, we introduce a global *Address Table*. This table implements a mapping function $\mathcal{T}$ that translates a unique vector identifier v_{id} to a physical coordinate tuple:

$$\mathcal{T}(v_{id}) \rightarrow \langle s_{id}, o_{slot} \rangle \quad (3)$$

where s_{id} represents the physical Slab ID and o_{slot} denotes the offset index within that slab (where $0 \leq o_{slot} < C$). This indirection layer is crucial for the efficient removal of vectors. Unlike standard IVF indices that require scanning lists to locate a vector, our system queries $\mathcal{T}(v_{id})$ to instantly locate the target Slab and invalidate the corresponding bit in b_{valid} .

3.1.4 System Implementation. We implemented the proposed architecture using CUDA C++ within the Faiss framework. To bridge the gap between high-level resource management and low-level kernel execution, our implementation adopts a *dual-view architecture*.

Host-Device Separation. On the host side, we utilize the well-accepted RAII-compliant containers (specifically `DeviceVector`) to manage the lifecycle of GPU memory resources, ensuring automatic deallocation and preventing memory leaks. These containers are encapsulated within a *SlabManager* class. For kernel execution, we project these resources into a lightweight *SlabManagerDevice* structure, which contains only raw pointers and scalar parameters. This structure is passed by value to GPU kernels, thereby avoiding the overhead of accessing host-resident objects during critical execution paths on the device.

Atomic State Transitions. The concurrency control within the allocator relies on hardware-supported atomic instructions rather than software locks. Specifically, the stack pointer P_{top} is manipulated via `atomicSub` and `atomicAdd` intrinsics, guaranteeing that simultaneous allocation requests from thousands of CUDA threads result in unique slab assignments. Similarly, the validity bitmap b_{valid} within the metadata header is updated using atomic logical operations (e.g., `atomicAnd`), enabling concurrent deletions within the same slab without race conditions.

Integration with Faiss. We integrated SDMA by extending the standard `GpuIndexIVF` class. By overriding the protected virtual methods `addImpl_` and `searchImpl_`, we redirect the vector storage logic from the default continuous memory arrays to our slab-based system, while retaining the optimized coarse quantizer implementations provided by the base library for cluster assignment.

3.2 Bitmap-based Lazy Eviction

[Dongfang: TODO]

4 Evaluation

Our implementation of SIVF is built upon the Faiss library [1], extending its GPU index classes to incorporate our proposed mechanisms. We conduct a comprehensive evaluation of SIVF against the native Faiss GPU index implementations, focusing on insertion and eviction performance under streaming workloads. All of our implementation and evaluation code are open-sourced as a side project to *ElasticIVF* hosted on GitHub¹.

5 Conclusion

In this paper, we identified a critical performance bottleneck in existing GPU-accelerated ANN indices stemming from the lack of native support for in-place data eviction. We analyzed the root causes of this limitation and proposed SIVF, a new indexing architecture that enables efficient streaming insertion and deletion directly on the GPU. By introducing slab-based memory management, bitmap-based lazy eviction, and a GPU-resident address translation table, SIVF overcomes the challenges of dynamic data structures in VRAM. Our evaluation demonstrates that SIVF significantly reduces eviction latency, paving the way for real-time streaming vector search applications on GPUs.

Acknowledgment

Results presented in this paper were obtained using the Chameleon testbed supported by the National Science Foundation.

References

- [1] DOUZE, M., GUZHYA, A., DENG, C., JOHNSON, J., SZILVASY, G., MAZARÉ, P.-E., LOMELI, M., HOSSEINI, L., AND JÉGOU, H. The faiss library.
- [2] JOHNSON, J., DOUZE, M., AND JÉGOU, H. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547.
- [3] JÉGOU, H., DOUZE, M., AND SCHMID, C. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128.
- [4] KEAHEY, K., ANDERSON, J., ZHEN, Z., RITEAU, P., RUTH, P., STANZIONE, D., CEVIK, M., COLLIERAN, J., GUNAWI, H. S., HAMMOCK, C., MAMBRETTI, J., BARNES, A., HALBACH, F., ROCHA, A., AND STUBBS, J. Lessons learned from the chameleon testbed. In *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC '20)*. USENIX Association, July 2020.

¹<https://github.com/hpdc/ElasticIVF>